🛡️ Cybersecurity & Ethical Hacking Internship Report

Organization: Ansh Infotech

Role: Cybersecurity Intern

Duration: January 2026 - Present

Focus: Linux Administration, Networking Fundamentals, Vulnerability Assessment, Social Engineering

📌 Executive Summary

This repository documents my daily learning progress, lab setups, and practical exercises during my internship. The training spans from core Linux system administration to offensive security techniques like social engineering and network exploitation.

📅 Phase 1: Linux Fundamentals & System Administration

Dates: Jan 15, 2026 – Jan 28, 2026

🔹 Learning Modules

1. **Linux Architecture:**
   - Explored distributions: **Kali Linux** (Offensive Security), **Ubuntu**, and **Debian**.
   - Understood the Linux File Hierarchy Structure (root /, /home, /etc, /var).
   - Desktop Environments.
2. **Essential CLI Commands:**
   - **File Ops:** touch (create), cat (read), cp (copy), mv (move), rm (remove).
   - **Editors:** Using nano for terminal-based text editing.
   - **System Info:** uname -a (kernel info), whoami, ls -h (human-readable list).
3. **User & Permission Management (Crucial for Security):**
   - **Permissions:** Mastered the **rwx** (Read-4, Write-2, Execute-1) system.
   - **Commands:**
     - chmod: Changing file modes (e.g., chmod +x script.sh to make executable).
     - chown: Changing file ownership.
     - sudo vs su: Superuser execution vs. switching user context.
4. **Package & Process Management:**
   - Managed software using apt (Advanced Package Tool): update, install, purge.

- **Monitoring:** Used top and ps to track real-time system resources.

📸 Practical Evidence

Practicing file permission commands and process monitoring.

Permission	Number
No permission	0
Execute (x)	1
Write (w)	2
Write & Execute (wx)	3
Read (r)	4
Read & Execute (rx)	5
Read & Write (rw)	6
Read, Write, & Execute (rwx)	7

Permissions	Alphabetic	Numeric	Description
---	`---`	0	No permissions.
--x	`--x`	1	Execute only.
-w-	`-w-`	2	Write only.
-wx	`-wx`	3	Write and execute.
r--	`r--`	4	Read only.
r-x	`r-x`	5	Read and execute.
rw-	`rw-`	6	Read and write.
rwx	`rwx`	7	Read, write, and execute.

📅 Phase 2: Vulnerability Assessment & Lab Setup

Date: Jan 29, 2026

🔹 Lab Configuration

Transitioned to an offensive mindset by setting up a safe, isolated environment for testing exploits.

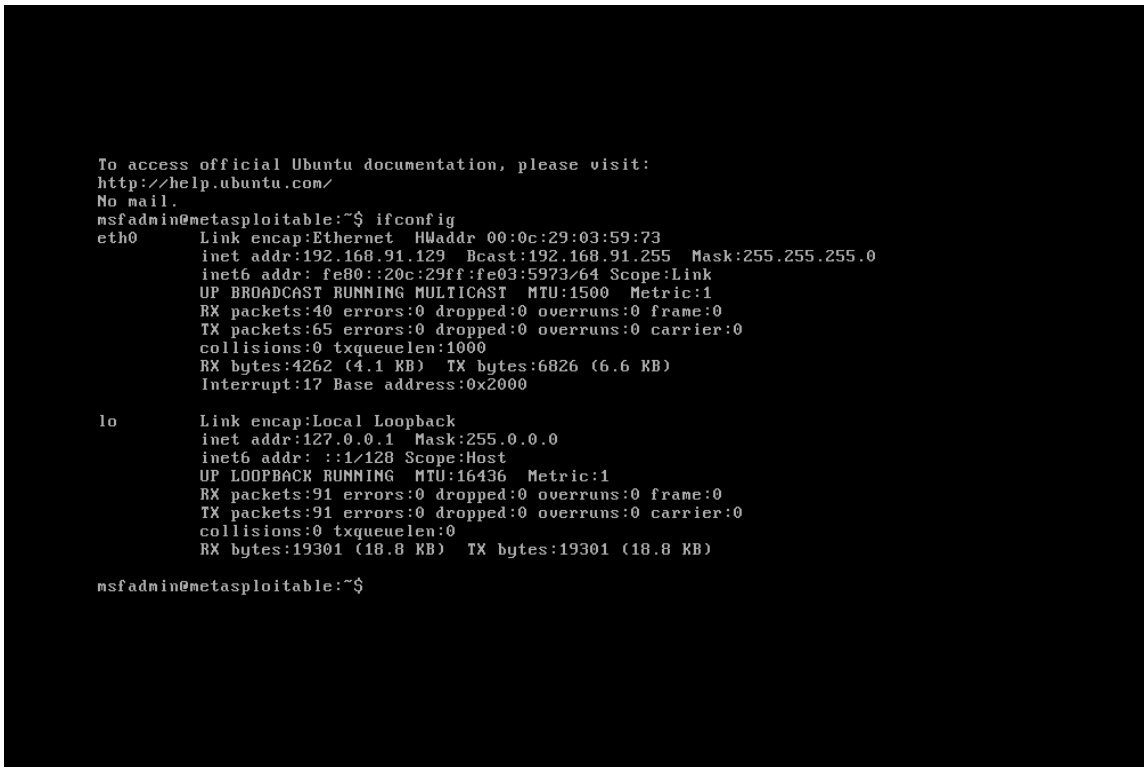
- **Target Machine:** Metasploitable (intentionally vulnerable Linux VM).
- **Attacker Machine:** Kali Linux.

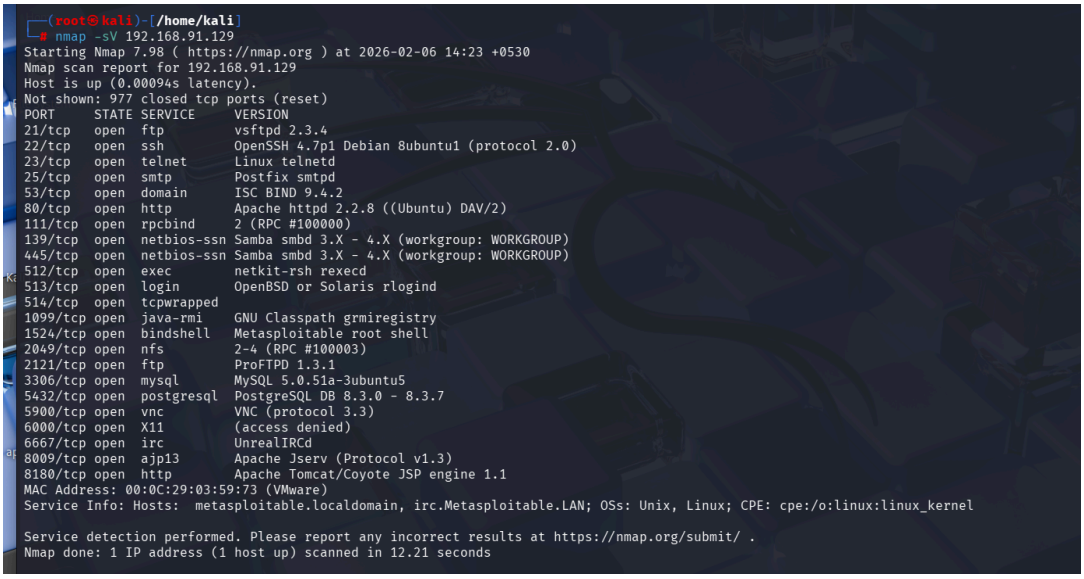
🔹 Tools & Techniques

- **NMAP (Network Mapper):**
  - Used for network discovery and security auditing.
  - Scanned targets to identify open ports and services.
- **Exploitation Basics:**
  - Connected to the vulnerable lab using **FTP** and **msfconsole** (Metasploit Framework).
- **Phishing Simulation:**
  - Deployed **Zphisher** to understand credential harvesting attacks.

📸 Practical Evidence

Setup of Metasploitable lab and initial port scanning.





📅 Phase 3: Networking Core Concepts

Dates: Jan 30, 2026 – Feb 4, 2026

🔹 Theory & Architecture

A solid understanding of networking is the backbone of cybersecurity.

1. **Network Topologies:**
   - **PAN:** Personal Area (Bluetooth/NFC).
   - **LAN/WLAN:** Local Area (Home/Office Wi-Fi).
   - **MAN/WAN:** Metropolitan & Wide Area (Inter-city/Internet).
2. **The OSI Model (7 Layers):**
   - Physical, Data Link, Network, Transport, Session, Presentation, Application.
   - Key Takeaway: Understanding where data exists (cable vs. IP packet vs. HTTP request) helps in diagnosing attacks.
3. **Addressing & Protocols:**
   - **MAC Address:** 48-bit physical hardware address (Hexadecimal).
   - **IP Address:** Logical address (IPv4 vs. IPv6).
   - **Transmission Modes:** Unicast (1-to-1), Multicast (1-to-many), Broadcast (1-to-all).
4. **Ports & Services Table:**
   - **FTP:** 20/21 (File Transfer)
   - **SSH:** 22 (Secure Shell)
   - **HTTP/HTTPS:** 80/443 (Web)
   - **DNS:** 53 (Domain Name System)

📅 Phase 4: Ethical Hacking - Social Engineering

Date: Feb 6, 2026

🔹 Human-Centric Vulnerabilities

Started the "Ethical Hacking" module, focusing on the weakest link in security: the human element.

🔹 Attack Vectors Studied

1. **Phishing:** Fraudulent communications to induce users to reveal sensitive info.
2. **Pretexting:** Fabricating scenarios (e.g., posing as HR or IT) to steal data.
3. **Baiting:** Using physical media (USB drives) or free downloads to entice victims.
4. **Tailgating/Piggybacking:** Following authorized personnel into restricted areas.
5. **Scareware:** Malicious software confusing users into visiting malware sites.

🔹 Tools Used

- **Zphisher:** For automated phishing templates.
- **ALHacking:** Comprehensive hacking tool suite.

📸 Practical Evidence

> Running Zphisher for educational phishing simulation.

📅 Phase 5: Real-World Reconnaissance & SSH Exploitation Testing

Dates: Feb 11, 2026 – Feb 12, 2026

🔹 Target Analysis & Information Gathering (Feb 11)

- **Target:** `gndec.ac.in (175.176.187.102)`
- **Reconnaissance:** Conducted active scanning using Nmap (`nmap -sV -sC`) to map out open ports, services, and OS fingerprints.
- **Findings:** Identified legacy software running on the live target, specifically OpenSSH 7.2p2 and Apache httpd 2.4.18, indicating an older Ubuntu 16.04 server environment.

🔹 Vulnerability Assessment & SSH Exploitation (Feb 12)

SSH Username Enumeration (CVE-2018-15473): * Attempted automated enumeration using Metasploit (`auxiliary/scanner/ssh/ssh_enumusers`).

- Identified that high network latency (2.2s) caused a "false positive storm" due to the

tool's integer-based threshold limitations.

Manual Timing Analysis: * Bypassed automated tool limitations by manually testing SSH response times (`time ssh`).

- Discovered an "inverted" timing pattern where valid users (e.g., `root`) responded significantly faster (0.52s) than nonexistent users (3.73s). Successfully confirmed the vulnerability with a clear ~3.2s timing delta.

SSH Brute-Force Testing: * Utilized Metasploit's `auxiliary/scanner/ssh/ssh_login` module to launch a dictionary attack against identified users.

- Monitored the server's response to high-frequency login attempts to evaluate the effectiveness of Intrusion Prevention Systems (e.g., Fail2Ban), noting the server permitted over 120 rapid sequential attempts without immediate IP blocking.

📅 Phase 6: Mobile Application Security & Static Analysis

Date: Feb 13, 2026

🔹 Mobile Security Assessment

Conducted a comprehensive Static Application Security Testing (SAST) audit on a third-party modified Android application to identify privacy violations, hardcoded secrets, and vulnerability patterns.

- **Target Application:** `InstaPro_v14.25.apk` (Modded Instagram Client)
- **Package Name:** `com.instapro.android`
- **Tool Used: MobSF (Mobile Security Framework) v4.4.5**

🔹 Key Findings (Static Analysis)

Security Score:

- The application received a **Security Score of 50/100 (Grade B)**, indicating Medium Risk.

Dangerous Permissions:

- Analysis revealed excessive permission requests that pose privacy risks, including:
  - `android.permission.READ_CONTACTS` (Read user contacts)
  - `android.permission.RECORD_AUDIO` & `CAMERA` (Surveillance risk)

    - `android.permission.READ_CALL_LOG` & `READ_SMS` (Sensitive communication data)
    - `android.permission.ACCESS_FINE_LOCATION` (Precise GPS tracking)

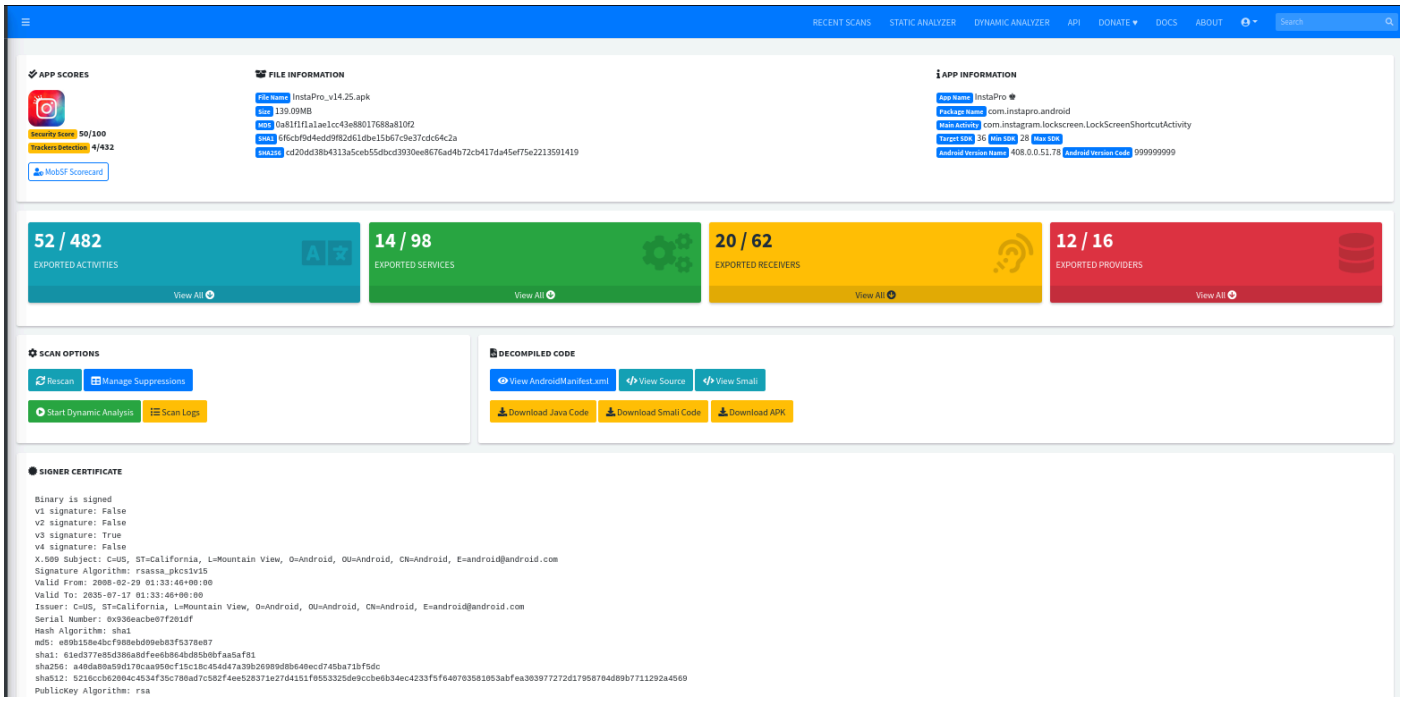
Hardcoded Secrets & Information Leakage:

- **API Keys:** Found exposed `google_api_key` and Firebase configuration URLs (`firebase_database_url`) in the source code.
- **Log Leakage:** Detected source code explicitly logging sensitive information, violating OWASP MSTG-STORAGE-3.

Code Vulnerabilities:

- **Cryptographic Weakness:** Identified the use of `CBC` encryption mode with `PKCS5/PKCS7` padding, which is vulnerable to padding oracle attacks (High Severity).
- **SQL Injection:** Found raw SQL queries executing untrusted user input (CWE-89).
- **Insecure Randomness:** Use of insufficient random number generators (CWE-330).

📸 Practical Evidence



Generated a full PDF report via MobSF highlighting the security score, tracker detection (4/432), and the manifest analysis of the `InstaPro` APK.

📅 Phase 7: Hands-on Exploitation (Hack The Box)

Date: Feb 16, 2026

🔹 CTF / Lab Challenges

Started practical machine exploitation on the Hack The Box (HTB) platform to reinforce scanning and enumeration skills.

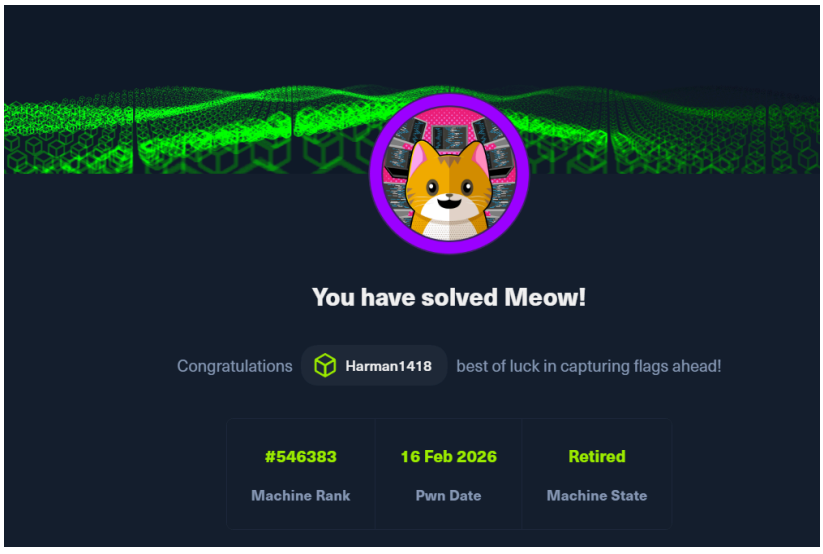
Target 1: Meow (Linux)

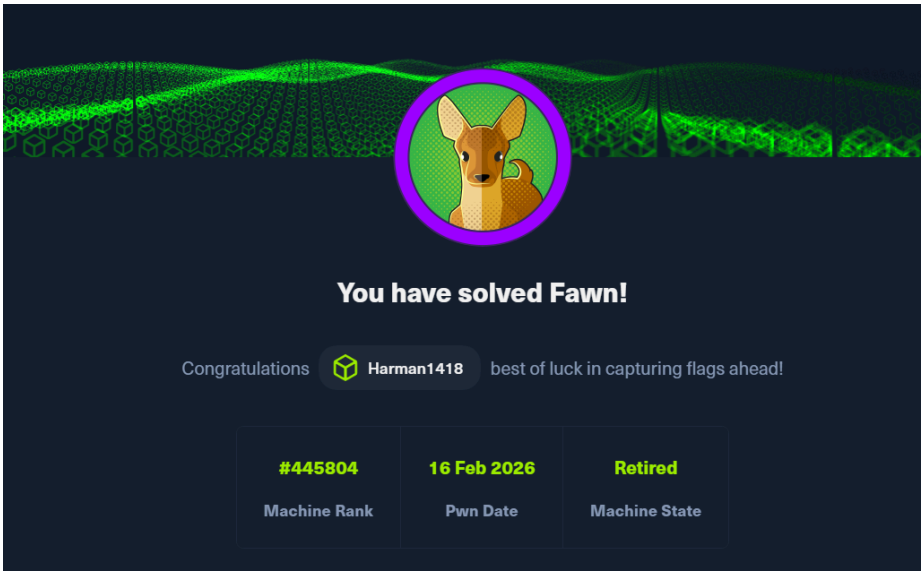
- **Difficulty:** Very Easy.
- **Objective:** Gain root access via Telnet.
- **Process:**
    - Connected via OpenVPN.
    - Scanned target using `nmap`. Identified Port 23 (Telnet) as open.
    - Attempted connection using `telnet <IP>`.
    - Exploited weak configuration by logging in as `root` with no password.
    - Retrieved the `root.txt` flag.

Target 2: Fawn (Linux)

- **Difficulty:** Very Easy.
- **Objective:** Data exfiltration via FTP.
- **Process:**
    - Scanned target using `nmap`. Identified Port 21 (FTP) as open.
    - Attempted `ftp <IP>` and successfully logged in using the `anonymous` user.
    - Used `ls` to list files and `get flag.txt` to download the flag.

📸 Practical Evidence





📅 Phase 8: Metasploitable Pentesting & Windows Setup

Date: Feb 17, 2026

🔹 Lab Penetration Testing (Metasploitable)

Continued penetration testing on the Metasploitable lab environment, focusing on remote access protocols.

Target: Metasploitable Linux VM. **Tools:** `msfconsole` (Metasploit Framework). **Protocols:** SSH (Port 22), Telnet (Port 23).

Exploitation Process:

- **Objective:** Perform dictionary attacks to gain credentials for remote access services.
- **Issue Encountered:** During the attack execution with `msfconsole`, the scanner failed to read the `user.txt` file correctly. The output showed: `[1] authentication failed ":<pass>"` instead of the expected format: `[1] authentication failed "<user>:<pass>"`
- **Troubleshooting & Fix:**
  - Analyzed the module configuration and input format.
  - **Solution:** Created a combined dictionary file named `userpass.txt` where usernames and passwords were separated by a space (format: `username password`).
  - **Result:** Re-running the attack with `userpass.txt` resolved the parsing error, allowing successful credential testing on both SSH and Telnet services.

🔹 Windows Environment Setup

- **Objective:** Prepare a secure testing ground for advanced post-exploitation techniques.
- **Action:** Successfully installed **Windows 11 Enterprise** on VMware.
- **Future Plan:** This environment will be used to test credential dumping tools like **Mimikatz** in an upcoming session.

📅 Phase 9: Post-Exploitation & Network Revision

Date: Feb 18, 2026

🔹 Advanced Credential Dumping (Mimikatz)

Focused on privilege escalation and credential extraction on Windows environments.

Windows 11 Enterprise (Attempt):

- **Objective:** Extract credentials from memory using Mimikatz.
- **Challenges:** Encountered enhanced security protections (Credential Guard/LSA Protection) preventing access.
- **Troubleshooting Steps:**
  - Attempted to run via `nsexec` to gain SYSTEM privileges.
  - Modified Registry permissions via `regedit` to disable LSA protections.
- **Outcome:** Despite troubleshooting, modern Windows 11 defenses prevented successful extraction in the lab environment.

Windows 10 Enterprise (Success):

- **Action:** Migrated testing to a Windows 10 Enterprise VM.
- **Outcome:** Successfully executed Mimikatz commands (`privilege::debug`, `sekurlsa::logonpasswords`) to retrieve NTLM hashes and cleartext credentials.

🔹 Network Security Revision

Revisited core networking concepts to solidify the theoretical foundation for advanced attacks.

- **Topics Reviewed:** OSI Model layers, common ports (21, 22, 80, 443, 445), and TCP/IP handshake.
- **Practical Exercise:** Conducted a Denial of Service (DoS) simulation using **hping3** to understand packet flooding mechanics (`hping3 -S --flood -V -p 80 <Target IP>`).

🔹 Homework: TryHackMe Lab

- **Module:** Post-Exploitation Basics.
- **Key Learnings:** Enumeration after compromise, maintaining access, and dumping system hashes using tools like **Hashcat** and **John the Ripper**.

📅 Phase 10: Web Application Logic Testing

Date: Feb 19, 2026

🔹 Live Target Vulnerability Assessment

Performed a business logic security test on a live e-commerce environment to identify parameter tampering vulnerabilities.

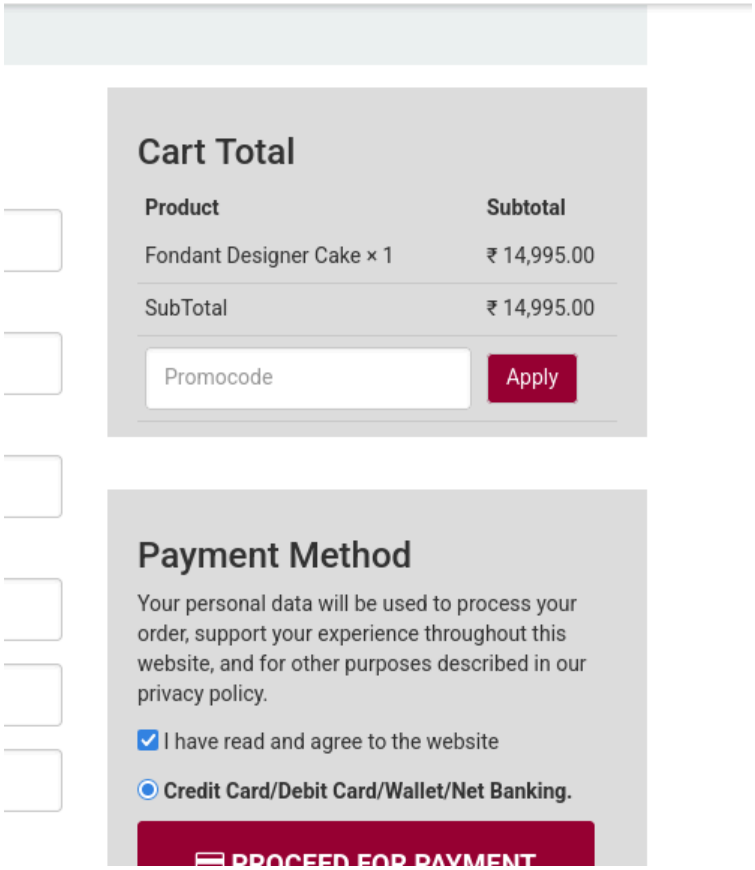
- **Target:** `www.ogdmart.com`
- **Vulnerability:** Parameter Tampering (Price Manipulation).
- **Tools Used:** Burp Suite (Proxy Interception).

🔹 Methodology

1. **Interception:** Configured browser proxy to route traffic through Burp Suite.
2. **Transaction Analysis:** Initiated a checkout process for a high-value item ("Fondant Designer Cake").
3. **Parameter Identification:** Analyzed the HTTP POST request body during the payment initiation phase.
    - **Original Value:** `cart_total=14995`
4. **Tampering:** Modified the `cart_total` parameter from `14995` to `5` in the intercepted request before forwarding it to the server.
5. **Verification:**
    - **Result:** The application accepted the modified value. The payment gateway page reflected the tampered price of **₹5** instead of the original **₹14,995**.
    - **Ethical Note:** The process was halted at the payment gateway page. No actual transaction was completed to strictly adhere to ethical hacking guidelines.

📸 Practical Evidence

Screenshots demonstrating the request interception and the reflected price change on the payment page.



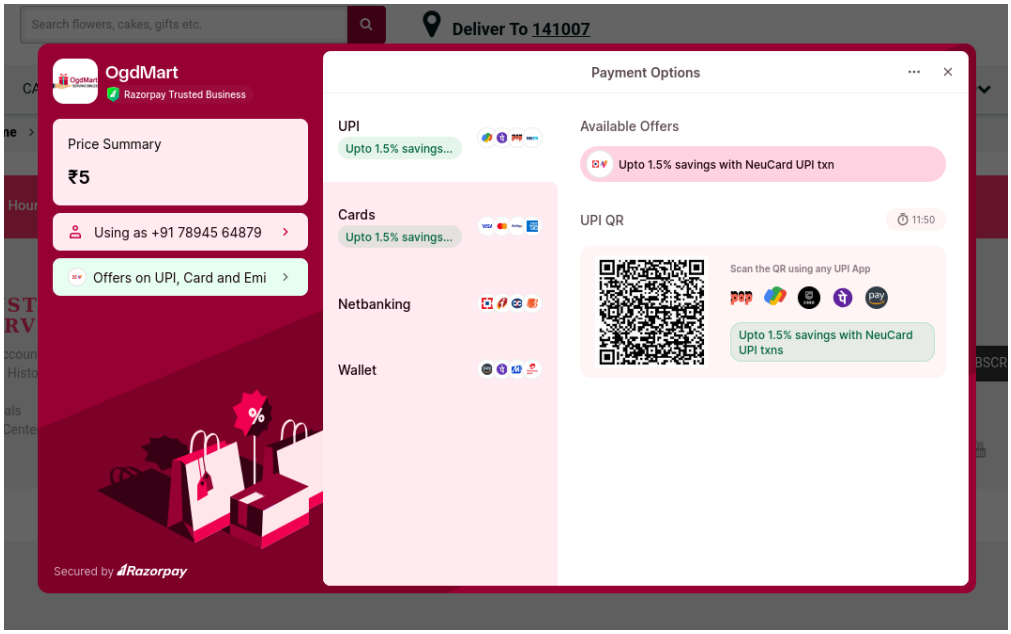
Before Tampering:

```
Te: trailers

txtBillingAddressFname=Burp&txtBillingAddressLname=Suite&txtBillingAddressEmailAddress=haaefa%40gmail.com&txtBillingAddressPhone=9876543210&txtBillingAddressCompany=&txtBillingAddressLine1=
11231%2Cdsfd%2Cdsfs&txtBillingAddressLine2=&txtBillingAddressLine3=&txtBillingTown=delhi&txtBillingState=delhi&txtBillingPostcode=141007&txtBillingCountry=INDIA&txtOrderNotes=&password=&
txtShippingAddressFname=Burp&txtShippingAddressLname=Suite&txtShippingAddressEmailAddress=haaefa%40gmail.com&txtShippingAddressPhone=9876543210&txtShippingAddressLine1=11231%2Cdsfd%2Cdsfs&
txtShippingAddressLine2=&txtShippingAddressLine3=&txtShippingTown=delhi&txtShippingState=delhi&txtShippingPostcode=141007&txtShippingCountry=INDIA&coupon_code=&cart_subt=14995&cart_disc=0&
cart_total=14995&paymenttype=2
```

After Tampering:

```
txtBillingAddressFname=Burp&txtBillingAddressLname=Suite&txtBillingAddressEmailAddress=haaefa%40gmail.com&txtBillingAddressPhone=9876543210&txtBillingAddressCompany=&txtBillingAddressLine1=
11231%2Cdsfd%2Cdsfs&txtBillingAddressLine2=&txtBillingAddressLine3=&txtBillingTown=delhi&txtBillingState=delhi&txtBillingPostcode=141007&txtBillingCountry=INDIA&txtOrderNotes=&password=&
txtShippingAddressFname=Burp&txtShippingAddressLname=Suite&txtShippingAddressEmailAddress=haaefa%40gmail.com&txtShippingAddressPhone=9876543210&txtShippingAddressLine1=11231%2Cdsfd%2Cdsfs&
txtShippingAddressLine2=&txtShippingAddressLine3=&txtShippingTown=delhi&txtShippingState=delhi&txtShippingPostcode=141007&txtShippingCountry=INDIA&coupon_code=&cart_subt=5&cart_disc=0&
cart_total=5&paymenttype=2
```



📅 Phase 11: Web Application Security & Environment Setup

Date: Feb 20, 2026

🔹 Live Target Assessment: OTP Verification

Conducted an assessment on a live financial portal to understand the implementation and potential weaknesses of One-Time Password (OTP) authentication mechanisms.

- **Target:** `www.instafinancials.com`
- **Vulnerability Category:** Broken Authentication / Business Logic Flaw (OTP Bypass)
- **Tools Used:** Burp Suite, FoxyProxy

🔹 Methodology & Exploitation Concept

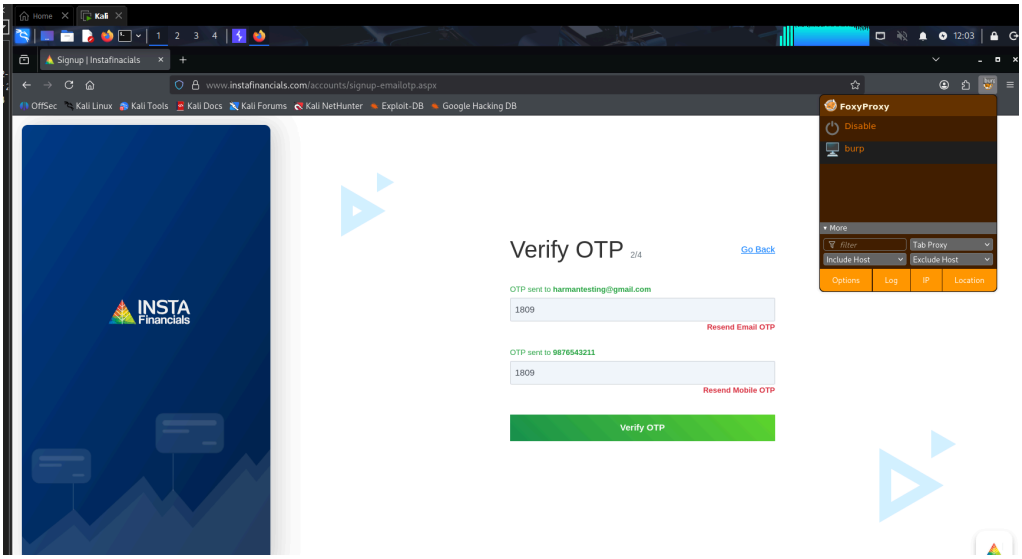
1. **Traffic Interception:** Configured FoxyProxy to route browser traffic through Burp Suite.
2. **Triggering Verification:** Initiated the account registration process and proceeded to the OTP verification step using testing data.
3. **Invalid Submission:** Entered an arbitrary, incorrect OTP (e.g., `1809`) to trigger a validation request.
4. **Response Manipulation:** Intercepted the server's response to the submitted OTP request. Modified the server response data in transit to simulate a successful

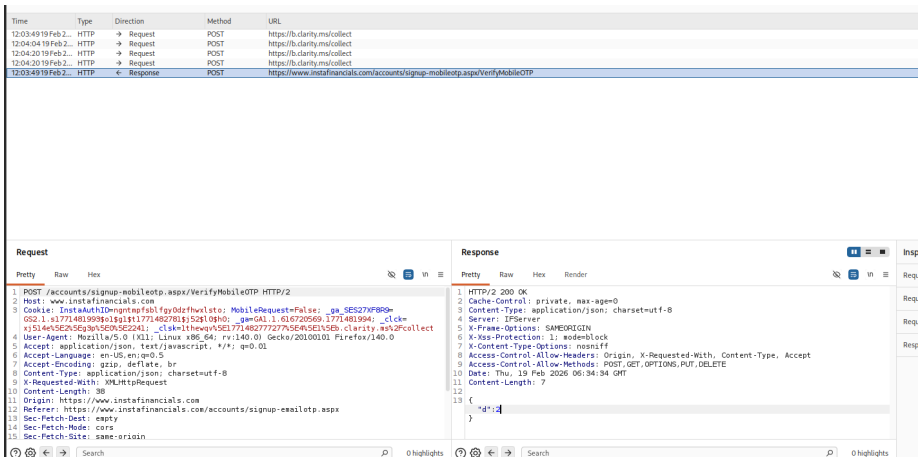
validation status.

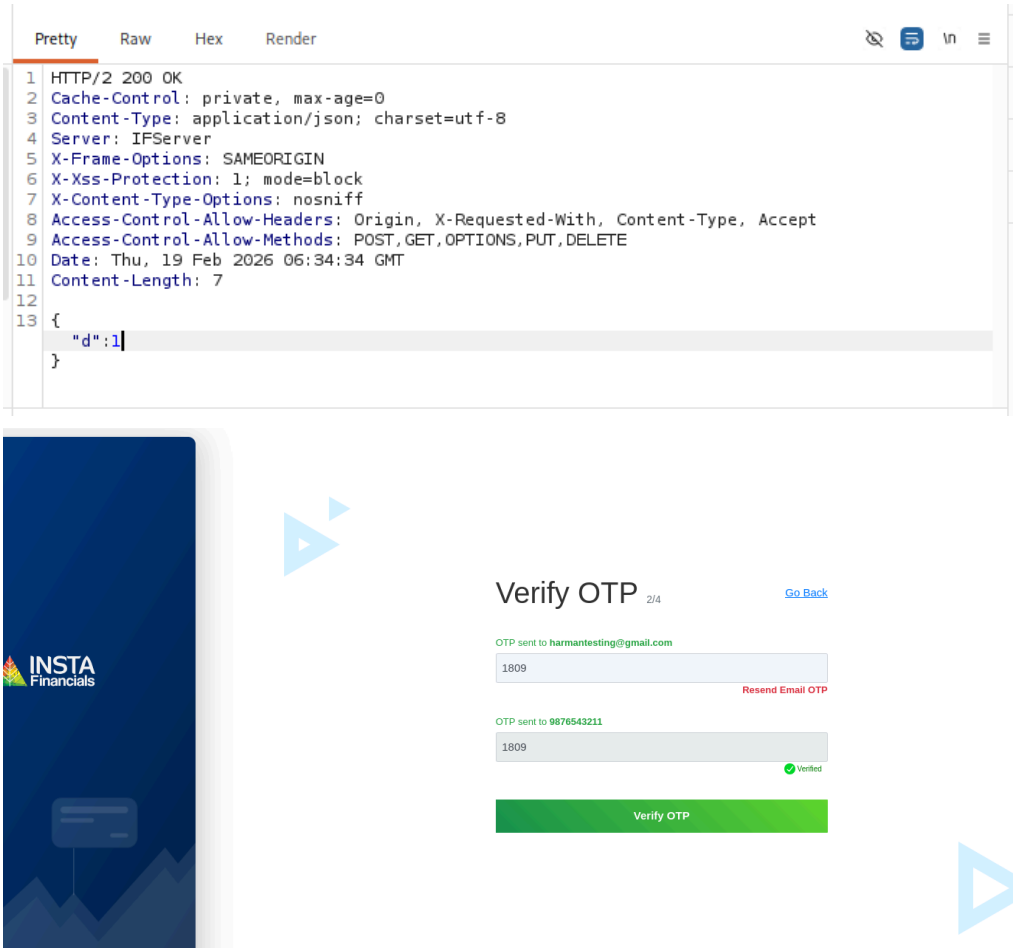
5. **Verification:** Forwarded the manipulated response back to the browser. The frontend application improperly trusted the modified response, bypassing the OTP check and granting access.
    - **Ethical Note:** Testing was conducted purely for educational purposes to demonstrate the risks of relying on client-side validation rather than strict server-side enforcement. No unauthorized access to sensitive user data was performed.

📸 Practical Evidence

Screenshots demonstrating the interception of the OTP request and the successful bypass via Burp Suite.







🔹 Environment Configuration: WSL & Kali Linux GUI

Expanded the local testing infrastructure by integrating Kali Linux directly into the Windows environment for streamlined access to penetration testing tools.

- **Technology Used:** Windows Subsystem for Linux (WSL 2), Win-KeX.
- **Process:**
  - **WSL Deployment:** Enabled the WSL feature on Windows and successfully installed the **Kali Linux** distribution natively.
  - **GUI Integration (Win-KeX):** To utilize graphical security tools seamlessly without a standalone VM, installed and configured **Win-KeX** (Kali Desktop Experience for Windows).
  - **Execution:** Initialized the graphical desktop environment using the `kex` command, establishing a persistent, low-overhead Kali GUI running parallel to the Windows host.
- **Outcome:** Achieved a fully functional, integrated Kali Linux desktop environment, significantly improving the speed and efficiency of the overall testing workflow.

📅 Phase 12: Active Reconnaissance & Portfolio Development

Dates: Feb 22, 2026 – Feb 23, 2026

🔹 Professional Portfolio Development (Feb 22)

- **Objective:** Establish a professional online presence to showcase cybersecurity projects, vulnerability assessments, and technical write-ups.
- **Action:** Developed and deployed a personal portfolio website hosted at `https://harmanjot-singh.vercel.app/`

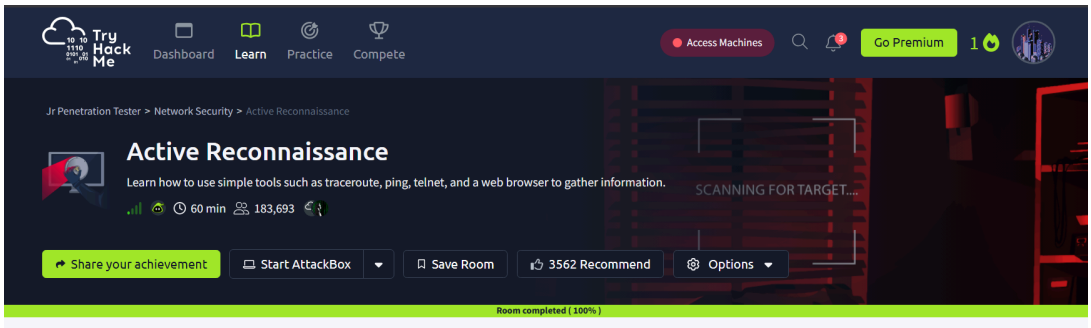
🔹 Active Reconnaissance Lab (Feb 23)

Completed the **Active Reconnaissance** room on the TryHackMe platform to reinforce network enumeration and information gathering techniques.

- **Tools & Commands Practiced:**
  - **`ping` / `ping -c` / `ping -n`:** Utilized ICMP echo requests to verify host availability on both Linux/macOS and Windows systems.
  - **`traceroute` / `tracert`:** Mapped the network path and routing hops to target systems to understand network topology.
  - **`telnet`:** Conducted manual port checking and banner grabbing on specific ports.
  - **`nc` (Netcat):** Practiced establishing connections as a client (`nc <IP> <PORT>`) and setting up listeners as a server (`nc -lvnp <PORT>`).
- **Outcome:** Achieved 100% completion of the room, successfully demonstrating the ability to gather actionable intelligence using simple, built-in system tools.

📸 Practical Evidence

TryHackMe room completion-



📅 Phase 13: Incident Response & Advanced Web Security

Date: Feb 24, 2026

🔹 Incident Response (TryHackMe)

Initiated the study of defensive cybersecurity frameworks, focusing on the first phase of the Incident Response lifecycle.

- **Module:** Incident Response - Phase 1: Preparation (TryHackMe).
- **Key Learnings:** * Understanding the core components of an Incident Response Plan (IRP).
    - The importance of establishing policies, training response teams, and preparing technical tools before a security breach occurs.
    - Evaluating business impact and configuring logging/monitoring solutions to ensure readiness.

🔹 Live Target Assessment: OTP Verification Flaw

Conducted a secondary real-world assessment to reinforce concepts around broken authentication and client-side trust issues.

- **Target:** `www.bakingo.com`
- **Vulnerability Category:** Broken Authentication / Business Logic Flaw (OTP Bypass)
- **Tools Used:** Burp Suite
- **Methodology:** 1. Triggered the OTP verification process during the login/registration flow. 2. Intercepted the validation request using Burp Suite Proxy. 3. Manipulated the server's response to the client, effectively forcing a "Success" condition on an invalid OTP entry. 4. The application improperly trusted the manipulated response and bypassed the authentication barrier.
- **Ethical Note:** This assessment was performed strictly for educational practice and concept reinforcement. The activity was halted post-verification, ensuring no unauthorized account takeovers or data alterations occurred.

🔹 Continuous Learning & Upskilling

To stay ahead of modern threat landscapes, began supplementing foundational knowledge with advanced paradigms.

- **Course:** "AI for CyberSecurity and Bug Bounty Hunting" (Udemy).
- **Objective:** Learn to leverage Artificial Intelligence, Large Language Models (LLMs),

and Machine Learning techniques to automate reconnaissance, identify complex vulnerability patterns, and optimize bug bounty workflows.

📅 Phase 14: Incident Response (Identification) & Network Traffic Analysis

Date: Feb 25, 2026

🔹 Incident Response: Identification and Scoping

Continued the Incident Response learning path, transitioning into the crucial second phase of the IR lifecycle.

- **Objective:** Identify indicators of compromise (IoCs) and scope the extent of an active incident.
- **Process:** * Analyzed multiple corporate email server logs to detect anomalous activities.
    - Successfully identified and isolated a spoofed phishing email that was engineered to look like it originated from the Chief Financial Officer's (CFO) account.
- **Key Learnings:** Understanding Business Email Compromise (BEC) tactics, analyzing email headers for spoofing, and scoping the potential blast radius of a phishing campaign. (Note: Further IR modules on this platform were paywalled, prompting a shift to Network Analysis).

🔹 Network Traffic Basics

Pivoted to analyzing the underlying data transmission that facilitates both attacks and normal operations across a network.

- **Module:** Network Traffic Basics
- **Tool Used: Wireshark** (Network Protocol Analyzer)
- **Process:**
    - Imported and analyzed provided PCAP (Packet Capture) files within Wireshark.
    - Applied display filters to isolate specific streams and protocols (e.g., HTTP, TCP, DNS) to cut through network noise.
    - Examined packet payloads and reconstructed TCP streams to find hidden data.
- **Outcome:** Successfully identified and extracted hidden flags embedded deep within the captured network packets, demonstrating proficiency in deep packet inspection (DPI) and traffic forensics.

📅 Phase 15: CTF Exploitation & Privilege Escalation

Date: Feb 26, 2026

🔹 Thundercipher CTF Labs

Completed two intermediate Capture The Flag (CTF) challenges on the Thundercipher platform, focusing on enumeration, steganography, and Linux privilege escalation using GTFOBins.

Target 1: Stegovault

- **Reconnaissance:** Conducted port scanning and service enumeration using `nmap`.
- **Enumeration:** Identified an open FTP service allowing anonymous login. Navigated the file system and successfully downloaded a suspicious `.jpeg` image file.
- **Exploitation (Steganography):** Leveraged `steghide` to extract hidden contents embedded within the image, which revealed valid SSH credentials.
- **Privilege Escalation:** Accessed the system via SSH. Identified that the current user could run the `docker` binary as root without a password. Exploited this misconfiguration by utilizing a `docker sudo` payload from GTFOBins (`sudo docker run -v /:/mnt --rm -it alpine chroot /mnt sh`) to break out of the restricted environment and spawn a root shell.

Target 2: Pagevault

- **Reconnaissance & Web Enumeration:** Utilized `nmap` for initial host discovery and `gobuster` for directory brute-forcing on the web server to uncover hidden paths.
- **Exploitation:** Discovered an exposed SSH private key during web enumeration. Performed URL decoding to properly format the key structure for authentication.
- **Privilege Escalation:** Successfully authenticated via SSH using the recovered private key. Discovered that the user had `sudo` rights to execute the `tar` command. Exploited this capability using a GTFOBins payload (`sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh`) to escalate privileges and obtain root access.

📅 Phase 16: Authentication Testing & SOC Incident Analysis

Date: Feb 27, 2026

🔹 Web Security: OTP Brute-Force via Sniper Attack

Explored the mechanics of automated credential stuffing and rate-limiting failures by testing an authentication mechanism using Burp Suite.

- **Concept Tested:** Insecure Authentication / Lack of Rate Limiting.
- **Tool Used: Burp Suite (Intruder)**
- **Methodology:**
    - Intercepted a mock OTP validation request using Burp Proxy.
    - Sent the request to **Burp Intruder** and configured the attack payload position specifically on the OTP parameter field.
    - Selected the **Sniper** attack type to test a single payload position systematically.
    - Loaded a customized payload list (sequential numbers/common OTPs) and initiated the automated attack to observe how the application handles rapid, repeated, incorrect submissions.
- **Key Learning:** Demonstrated the critical importance of implementing account lockouts, exponential backoff delays, or CAPTCHA validation to protect against automated brute-force attacks on OTP endpoints.

🔹 SOC / Incident Response: TryHackMe Carnage Lab

Completed the "Carnage" room on TryHackMe, focusing on Blue Team operations, malware traffic analysis, and forensic investigation.

- **Objective:** Act as a SOC Analyst to investigate a simulated corporate network compromise involving malicious email attachments and malware execution.
- **Tools Used: Wireshark**
- **Process:**
    - Analyzed a large PCAP file containing the network traffic of the infected workstation.
    - Filtered traffic to isolate the initial infection vector (e.g., identifying the download of the malicious payload from a suspicious domain).
    - Extracted Indicators of Compromise (IoCs), including malicious IP addresses, domain names, and the specific malware executable signature.
    - Traced the timeline of the attack, from the initial HTTP GET request to subsequent Command and Control (C2) communications.
- **Outcome:** Successfully mapped out the kill chain of the attack, reinforcing practical skills in network forensics and incident triage.

📅 Phase 17: Cryptography & Automated Vulnerability Scanning

Dates: Mar 01, 2026 – Mar 10, 2026

🔹 Applied Cryptography & Encoding Techniques

Expanded knowledge into cryptography, learning how data is encrypted, encoded, and obfuscated to hide information or secure communications.

- **Concepts Practiced:**
  - Encoding mechanisms: **Base32, Base64, Decimal, HEX, Binary**.
  - Substitution ciphers: **ROT13, ROT47**.
  - Audio/Visual encoding: **Morse Code**.
- **Practical Application:** Completed the TryHackMe room `c4ptur3th3fl4g`.
  - Objective: Translate and decode various obfuscated strings and messages using the appropriate cryptographic or encoding algorithms.
  - Outcome: Gained a solid understanding of how to quickly recognize data structures (e.g., padding characters in Base64, hex structures) to decrypt/decode payloads during penetration tests.

🔹 Automated Vulnerability Scanning (Live Target)

Transitioned to using industry-standard automated vulnerability scanners to audit live network environments.

- **Target:** `gndec.ac.in` (IP: `175.176.187.102`)
- **Objective:** Perform broad-spectrum vulnerability discovery to complement previous manual enumeration.

Tool 1: Nessus (Tenable)

- Conducted a basic network scan against the target IP.
- **Findings:** Identified 2 INFO-level vulnerabilities (Nessus Scan Information and Ping the remote host). The scan confirmed basic host availability and network responsiveness.

Tool 2: Pentest-Tools (Light Website Scanner)

- Conducted a web vulnerability scan against `https://gndec.ac.in/`.
- **Findings:** The scanner assessed the overall risk level as **High**.
  - **Critical:** 0 | **High:** 2 | **Medium:** 4 | **Low:** 7
  - **Key Discovery:** Identified vulnerabilities tied to the underlying `Apache HTTP`

`Server 2.4.18`. Specifically flagged the target for CVEs such as **CVE-2024-38476**, indicating susceptibility to information disclosure, Server-Side Request Forgery (SSRF), or local script execution due to the legacy web server version.

📅 Phase 18: Web Security Concepts & Tool Development

Date: Mar 11, 2026

🔹 Web Application Security: Content Security Policy (CSP)

Explored the mechanisms of modern web application defense layers.

- **Concept Studied:** Deep dive into Content Security Policy (CSP).
- **Key Learnings:** * How CSP acts as an added layer of security to detect and mitigate specific attack vectors, including Cross-Site Scripting (XSS) and data injection attacks.
    - Understanding the implementation of CSP directives (e.g., `default-src`, `script-src`, `style-src`) to restrict the load origins of executable scripts and external resources.
    - Recognized how missing or misconfigured CSP headers leave web applications vulnerable to unauthorized code execution.

🔹 Project Initiation: VulnForge

Began the development of a custom cybersecurity utility to automate reconnaissance workflows.

- **Project Name: VulnForge**
- **Objective:** Build an automated network enumeration and Vulnerability Assessment and Penetration Testing (VAPT) web platform.
- **Inspiration:** Designed to mimic the workflow of professional automated scanners like `pentest-tools.com`.
- **Planned Architecture & Features:**
    - **Automated Reconnaissance:** Integration of backend port scanning tools to seamlessly identify open network ports and active services.
    - **Vulnerability Scanning:** Implementing automated checks against target environments to detect known misconfigurations or outdated software.
    - **Web Interface:** Constructing a user-friendly dashboard to initiate scans, parse raw tool outputs, and display structured risk assessment reports to users.

📅 Phase 19: Comprehensive External VAPT Assessment

Date: Mar 12, 2026

🔹 Live Target Penetration Testing

Conducted a full-scale external Vulnerability Assessment and Penetration Testing (VAPT) engagement on a live infrastructure target to identify security flaws, misconfigurations, and outdated components.

- **Target:** `anc.org.au` (IP: `101.0.69.242`)
- **Objective:** Perform non-destructive testing to evaluate the external security posture and compile a professional-grade confidential VAPT report.

🔹 Assessment Methodology & Tools Used

- **Reconnaissance & Scanning:** Utilized **Nmap** (v7.98) for initial host discovery and vulnerability scanning across the external perimeter.
- **Web Misconfiguration Scanning:** Deployed **Nikto** (v2.5.0) and **Pentest-Tools** to scan the web server for known vulnerabilities and missing security headers.
- **Cryptographic Auditing:** Used **ssh-audit** (v3.3.0) and **OpenSSL** (`s_client`) to manually verify TLS and SSH cryptographic configurations.
- **SMTP Enumeration:** Leveraged **swaks** (Swiss Army Knife for SMTP) for testing STARTTLS and mail server enumeration.

🔹 Key Findings & Vulnerabilities

- **Exposed Administrative Interfaces:** Discovered critical management ports exposed to the public internet, including SSH running on a non-standard port (`6969`), cPanel (`2078`/`2083`), and FTP (`21`).
- **Outdated Services:** Identified legacy versions of OpenSSH, Exim, and Apache requiring immediate patching.
- **Cryptographic Weaknesses:** Audits revealed the support of weak, deprecated cryptographic algorithms, including SHA-1, 3DES, and Anonymous Diffie-Hellman.
- **Web Application Risks:** Detected missing `HttpOnly` and `Secure` flags on application session cookies, and the absence of a strict Content-Security-Policy (CSP).

🔹 Reporting & Remediation Strategy

Compiled a comprehensive, heavily restricted VAPT report detailing the attack narrative, threat model, and a structured remediation plan:

- **30-Day Action Plan:** Restrict administrative ports (6969, 2078/2083, 21) to trusted

VPN/IP subnets via hardware firewall, upgrade core services (OpenSSH, Exim, Apache), and secure session cookies with proper flags.
- **90-Day Action Plan:** Enforce modern TLS/SSH configurations globally (explicitly disabling weak ciphers) and implement a strict CSP alongside upgrading client-side dependencies (jQuery, Bootstrap).

📅 Phase 20: Advanced Web Application Attacks

Dates: Mar 13, 2026 – Mar 14, 2026

🔹 In-Depth Study: Web Cache Poisoning

Explored the complexities of web caching mechanisms and how they can be manipulated to serve malicious content to legitimate users.

- **Concept Studied:** Web Cache Poisoning.
- **Key Learnings:** * Understanding how reverse proxies and Content Delivery Networks (CDNs) cache responses based on "keyed" components (like the URL and Host header).
    - Identifying "unkeyed" inputs (e.g., `X-Forwarded-Host`, `X-Original-URL`, or custom headers) that are processed by the backend application but ignored by the cache key.
    - Learning how to inject malicious payloads (such as Cross-Site Scripting or open redirects) via unkeyed headers, forcing the cache server to save and serve the exploited response to all subsequent visitors.

🔹 In-Depth Study: Insecure Deserialization

Investigated one of the most critical and complex web vulnerabilities that often leads to Remote Code Execution (RCE).

- **Concept Studied:** Insecure Deserialization (OWASP Top 10).
- **Key Learnings:**
    - Understanding the process of serialization (converting application objects into data formats like JSON, XML, or binary) and deserialization (reconstructing objects from data).
    - Recognizing how untrusted, user-controllable serialized data can be maliciously modified before being processed by the backend (e.g., PHP Object Injection, Python Pickle exploits, Java deserialization).
    - Exploring the severe impact of deserialization flaws, ranging from authentication bypass and privilege escalation to complete server compromise via RCE.
    - Emphasizing mitigation strategies, such as avoiding the deserialization of

untrusted data entirely or using digital signatures/integrity checks to prevent tampering.

📅 Phase 21: Tool Development - VulnForge Architecture & AI Integration

Date: Mar 15, 2026

🔹 Frontend Development (Completed)

Finalized the user interface and client-side logic for the custom VAPT platform, VulnForge. Focus was placed on creating a responsive, hacker-themed dashboard without relying on heavy external UI frameworks like Tailwind or Bootstrap.

- **Core Stack:** React 18 built via Vite.
- **Routing & APIs:** React Router v6 for page navigation and Axios for making asynchronous API calls to the backend.
- **Styling & Typography:** Custom Inline CSS paired with thematic Google Fonts (Orbitron, Share Tech Mono, Rajdhani).
- **Deployment:** Hosted securely on the Vercel cloud platform.

🔹 Backend & Real-Time Engineering (In Progress)

Initiated the backend infrastructure to handle intensive scanning logic, continuous data streaming, and the future integration of Artificial Intelligence.

- **Core Framework:** Python 3.12 utilizing the FastAPI web framework and Uvicorn ASGI server.
- **Real-Time Features:** Integrated WebSockets to stream live VAPT scan progress and port scanning results directly to the client interface.
- **Infrastructure:** Deployed on a Microsoft Azure Virtual Machine (B2ats v2 instance running Ubuntu 24.04).
- **AI Integration:** Actively working to embed an AI analysis layer to automatically parse raw scan logs and highlight actionable vulnerabilities.

🔹 Infrastructure & Email Integration

Secured external resources and integrated essential communication APIs for application deployment.

- **Domain Acquisition:** Successfully registered and configured the official production domain `vulnforge.app`.

- **Transactional Email Service:** Integrated the **Resend API** (`resend.com`) into the backend architecture.
- **OTP Delivery:** Set up the verified sender address `noreply@vulnforge.app` to seamlessly handle the secure dispatch of One-Time Passwords (OTPs) for the platform's user registration and authentication flows.

🔹 Database Architecture

Designed an asynchronous, cloud-native database schema to handle user authentication, target tracking, and historical scan logs.

- **Provider:** MongoDB Atlas (Cloud Database - Free Tier).
- **Driver Integration:** Motor (Asynchronous MongoDB driver for Python).
- **Collections Defined:**
  - `users`: User authentication and profile management.
  - `otps`: Secure email verification and password reset handling.
  - `scans`: Historical logging of all VAPT scans.
  - `targets`: Management of user-defined IP addresses and domains to be scanned.

📅 Phase 22: Practical Exploitation - Deserialization & Cache Poisoning

Dates: Mar 16, 2026 – Mar 17, 2026

🔹 Hands-On: Web Cache Poisoning

Transitioned from theoretical study (Phase 20) to practical exploitation of caching mechanisms.

- **Tools Used:** Burp Suite Professional, **Param Miner** extension.
- **Process:** * Actively probed web applications to identify unkeyed HTTP headers (e.g., `X-Forwarded-Host`, `X-Original-URL`).
  - Injected malicious Cross-Site Scripting (XSS) payloads into the unkeyed headers.
  - Forced the caching server (like Varnish or Cloudflare) to store the manipulated response and serve it to subsequent legitimate users.

🔹 Hands-On: Insecure Deserialization

Practiced identifying and exploiting object serialization flaws to achieve Remote Code

Execution (RCE).

- **Tools Used:** Burp Suite, **Ysoserial**.
- **Process:**
    - Intercepted HTTP requests to identify serialized data structures passed in cookies or API endpoints (e.g., base64-encoded Java objects starting with `r00`, or PHP serialized strings).
    - Utilized the Ysoserial tool to generate custom malicious gadget chains designed to execute system commands.
    - Replaced the legitimate serialized objects with the malicious payloads to trigger RCE upon server-side deserialization.

📅 Phase 23: Automation & Web Vulnerabilities (TryHackMe)

Dates: Mar 18, 2026 – Mar 20, 2026

🔹 Shell Scripting & Automation

Transitioned into building automated workflows to streamline enumeration and reconnaissance processes using the Linux command line.

- **Module:** Bash Scripting (TryHackMe)
- **Key Learnings:**
    - Mastered Bash fundamentals: declaring variables, passing parameters, and utilizing arrays.
    - Implemented control flow using conditional statements (`if/elif/else`) and logic loops (`for/while`).
    - Developed foundational utility scripts to automate repetitive command-line tasks, significantly reducing manual overhead during footprinting and initial access phases.

🔹 Web Application Vulnerabilities: XSS

Deepened web application testing capabilities by exploring client-side injection vulnerabilities.

- **Module:** Cross-Site Scripting / XSS (TryHackMe)
- **Key Learnings:**
    - **Reflected XSS:** Exploited non-persistent injection points where malicious scripts are bounced off the web server (e.g., via search parameters or error messages).

    - **Stored XSS:** Identified and exploited persistent vulnerabilities where payloads are saved to the database (e.g., comment sections, user profiles) and executed upon victim interaction.
    - **DOM-Based XSS:** Analyzed client-side JavaScript execution flaws altering the Document Object Model without direct server interaction.
    - **Payload Crafting & Mitigation:** Practiced crafting custom JavaScript payloads (e.g., `<script>alert(1)</script>`, cookie stealing) and reviewed mitigation strategies such as strict output encoding and input validation.

📅 Phase 24: Cloud Infrastructure & Secure Deployment

Date: Mar 21, 2026

🔹 AWS Portfolio Deployment

Transitioned personal projects to cloud infrastructure, focusing on secure server provisioning and access management.

- **Objective:** Host a personal portfolio website securely on public cloud infrastructure.
- **Platform Used:** Amazon Web Services (AWS)
- **Process & Security Implementation:**
    - Provisioned an **Ubuntu Linux EC2 virtual machine** within an AWS Virtual Private Cloud (VPC).
    - **Network Hardening:** Configured AWS Security Groups (virtual firewalls) following the principle of least privilege. Explicitly allowed inbound traffic only for necessary web services (HTTP on Port 80, HTTPS on Port 443).
    - **Access Control:** Secured remote administration by restricting SSH (Port 22) access. Disabled password-based logins entirely and enforced strict RSA key-pair authentication.
    - Successfully deployed the portfolio web application assets to the secured Ubuntu environment.

📅 Phase 25: Tool Development - Infrastructure Scaling & Optimization

Date: Mar 22, 2026

🔹 VulnForge Deep Scan Implementation & Resource Bottlenecks

Continued backend development on the VulnForge VAPT platform, specifically focusing on the

execution of deep vulnerability scans.

- **Challenge Encountered:** During the testing of comprehensive "deep scans" (running multiple concurrent enumeration and scanning tools simultaneously), the backend service became unresponsive and stuck in a persistent loop.
- **Root Cause Analysis:** Diagnosed the issue as severe resource exhaustion (Out of Memory / OOM error). The current Microsoft Azure Free Tier Virtual Machine (B2ats v2 instance, featuring 1GB RAM) provided insufficient memory to handle the heavy concurrent processing demanded by deep security scanners.
- **Proposed Resolution & Next Steps:** Initiated a migration plan to move the backend infrastructure to a significantly more robust environment. Currently provisioning an **Oracle Cloud Infrastructure (OCI) Always Free Tier** instance, which offers up to **24GB of RAM and 4 ARM Ampere A1 Compute OCPUs**. This massive upgrade will provide the necessary computational power and memory footprint to execute concurrent VAPT scans without throttling or crashing.

📅 Phase 26: Layer 2 Network Attacks & VulnForge Enhancements

Date: Mar 23, 2026

🔹 Network Security: Layer 2 Attacks (TryHackMe)

Expanded practical knowledge of localized network exploitation focusing on Layer 2 vulnerabilities.

- **Module:** MAC Flooding & ARP Spoofing (TryHackMe)
- **Key Learnings:**
    - **MAC Flooding:** Explored how overwhelming a network switch's CAM table forces it to default to a "hub" mode, broadcasting all network traffic to all ports and enabling unauthorized packet sniffing.
    - **ARP Spoofing (Poisoning):** Learned the mechanics of sending forged Address Resolution Protocol (ARP) messages to associate an attacker's MAC address with a legitimate IP address (like a default gateway). This effectively facilitates Man-in-the-Middle (MitM) attacks, allowing interception, modification, or dropping of local network traffic.

🔹 Tool Development: VulnForge Platform Upgrades

Iterated on the VAPT platform to significantly enhance user experience and scanning depth.

- **Frontend UI/UX Enhancements:** Integrated fluid UI animations into the React

interface. This improves visual feedback for the user during long-running tasks, particularly during the transition states of vulnerability scans.
- **Backend Tool Integration:** Successfully mapped and integrated a new suite of security tools into the backend engine, expanding the platform's capacity to identify a wider range of network and web application vulnerabilities.

🔹 Infrastructure Update: Oracle Cloud Migration

- **Status:** Actively attempting to procure and configure the Oracle Cloud Infrastructure (OCI) Free Tier instance. The goal remains to leverage the 24GB RAM and 4 OCPU capacity to permanently resolve the scanning bottlenecks experienced on the previous Azure architecture.

📅 Phase 27: Android VAPT & Reverse Engineering

Date: Mar 24, 2026

🔹 Mobile Application Security Analysis (Spotify APK)

Conducted a comprehensive security audit and reverse engineering exercise on a high-complexity Android application to identify local vulnerabilities and analyze network communication.

1. Decompilation & Static Analysis:

- **Deconstruction:** Utilized `apktool` to decode the Spotify APK, converting compiled `.dex` files into human-readable **Smali** code and extracting the `AndroidManifest.xml`.
- **Manifest Audit:** Identified the application's attack surface by auditing **Exported Activities**, **Intent Filters**, and custom permissions.
- **Permission Analysis:** Evaluated the application's extensive access to sensitive hardware and data, including Precise Location, Bluetooth, and Audio Recording.

2. Security Patching & Rebuilding:

- **SSL Pinning Bypass:** Manually modified the `network_security_config.xml` to inject a custom security policy. This patch allowed the application to trust a User-added CA (Burp Suite) certificate, effectively bypassing the certificate pinning protections standard on Android 7.0+.
- **Resource Optimization & Fixes:** Resolved build-time errors by identifying and stripping unsupported Android 15 attributes (e.g., `android:isCredential`) from the XML resources using `sed`.
- **Binary Integrity:** Recompiled the modified source code and signed the resulting APK

with a custom RSA 2048-bit key using `keytool` and `apksigner`.

3. Dynamic Analysis & Network Interception:

- **Environment Isolation:** Deployed the patched APK into a rooted **Genymotion** emulator to isolate testing from the host environment.
- **Intent Fuzzing:** Performed dynamic stress testing by triggering automated Android Intents via `adb shell am start` to verify the robustness of exported components.
- **Log Monitoring:** Utilized `adb logcat` with precise error filters to monitor real-time Java exceptions and background application logic during runtime.
- **MITM Architecture:** Established a persistent Man-in-the-Middle (MITM) setup by routing all emulator traffic through Burp Suite using an **ADB Reverse Tunnel** (`adb reverse tcp:8080 tcp:8080`).

🔹 Summary of Findings

The application demonstrated high resilience against basic intent fuzzing. While standard system proxy settings were initially bypassed by the app's custom networking core, the manual injection of the `network_security_config.xml` successfully established the environment required for deep API and traffic inspection.

📅 Phase 28: Advanced APK Manipulation & Active Directory Exploitation

Date: Mar 25, 2026

🔹 Advanced Mobile Reverse Engineering

Deepened practical knowledge of Android application modification and repackaging.

- **Tool Used:** `apktool`
- **Key Learnings:** * Explored the structure of decompiled Android applications, specifically focusing on the `AndroidManifest.xml` framework.
    - Practiced manually adding, removing, and modifying application permissions (e.g., granting internet access or storage permissions to restricted apps).
    - Repackaged and signed the modified applications to observe how altered permissions directly affect runtime behavior, system access, and security boundaries.

🔹 Active Directory Exploitation Lab (TryHackMe)

Expanded network exploitation skills into enterprise Windows environments by exploiting a

recent Active Directory vulnerability.

- **Vulnerability Studied:** CVE-2022-26923 (Active Directory Domain Services Elevation of Privilege)
- **Lab Environment:** TryHackMe CVE-2022-26923 (Certifried)
- **Methodology & Key Learnings:**
    - Analyzed the mechanics of **Active Directory Certificate Services (AD CS)** and the processes involved in issuing certificates to domain computers.
    - Exploited a flaw in how AD CS validates the `dNSHostName` attribute of a newly created or modified machine account.
    - Successfully spoofed a Domain Controller by requesting a machine certificate using a crafted `dNSHostName`, allowing for a massive escalation of privileges from a standard domain user to a highly privileged enterprise administrative account.

📅 Phase 29: Intrusion Detection Systems & SQL Injection

Date: Mar 26, 2026

🔹 Infrastructure Setup: Snort IDS Deployment

Focused on defensive network security by deploying and configuring a robust Intrusion Detection System (IDS).

- **Environment:** Provisioned a local **Ubuntu 24.04** environment specifically tailored for network monitoring and defense.
- **Tool Deployed: Snort** (Network Intrusion Detection and Prevention System).
- **Key Learnings:** * Installed and configured Snort to monitor network traffic in real-time.
    - Explored the mechanics of writing and implementing custom Snort rules to detect malicious network signatures, port scans, and anomalous traffic patterns.

🔹 Web Application Vulnerabilities: SQL Injection (SQLi)

Returned to offensive web application testing to master database-layer attacks.

- **Module:** SQL Injection / `sqli` lab (TryHackMe).
- **Key Learnings:**
    - **In-Band SQLi:** Practiced exploiting Error-Based and Union-Based SQL injections to extract database schema and user credentials directly within the

web application's response.
    - **Blind SQLi:** Explored Boolean-Based and Time-Based Blind SQL injections, learning how to infer database structures by analyzing server response times and conditional true/false HTTP responses.
    - **Mitigation Strategy:** Reviewed defensive coding practices, specifically the implementation of Prepared Statements (Parameterized Queries) to neutralize SQLi vectors.

📅 Phase 30: Intrusion Detection System (IDS) Implementation

Date: Mar 27, 2026

🔹 Practical Snort Implementation & Configuration

Transitioned from the initial installation phase to the active configuration and implementation of Snort IDS within the local Ubuntu environment.

- **Network Configuration:** Modified the primary `snort.conf` file to accurately define network boundaries. Configured the `HOME_NET` variable to represent the internal subnet being protected and the `EXTERNAL_NET` variable to represent all outside traffic, ensuring precise classification.
- **Custom Rule Creation:** Developed custom detection signatures within the `local.rules` file to identify specific network anomalies and common attack vectors. Examples included rules to flag ICMP Echo (Ping) sweeps, SSH brute-force attempts, and Nmap stealth/XMAS scans.
- **Traffic Analysis & Alerting:** Ran Snort in IDS mode (e.g., `snort -A console -q -c /etc/snort/snort.conf -i eth0`) and simulated external attacks using Kali Linux tools. Successfully verified that custom rules correctly triggered real-time alerts and populated the `/var/log/snort/alert` file for further incident response analysis.

📅 Phase 31: Advanced Intrusion Detection & Backend Integration

Date: Mar 28, 2026

🔹 Deep Dive: IDS/IPS Architecture (TryHackMe)

Transitioned from local implementation to advanced scenario-based analysis using TryHackMe's dedicated Snort lab.

- **Module:** Snort (TryHackMe)
- **Key Learnings:**
    - Explored the distinct architectural and operational differences between passive network monitoring (Intrusion Detection Systems - IDS) and active threat blocking (Intrusion Prevention Systems - IPS).
    - Practiced analyzing malicious Packet Captures (PCAPs) directly against custom Snort rulesets.
    - Fine-tuned signature logic to increase detection accuracy while proactively minimizing false positive alerts in high-traffic environments.

🔹 Tool Development: VulnForge Backend Engineering

Continued intensive backend development for the VulnForge VAPT platform, focusing on API optimization and asynchronous data handling.

- **API Optimization:** Enhanced the FastAPI application routing to better handle concurrent scanning queues and WebSockets data streams.
- **Database Integration:** Optimized MongoDB read/write operations using the asynchronous Motor driver. Ensured that real-time scan metrics, vulnerability logging, and user state updates are written non-blockingly, keeping the main execution thread highly performant.

📅 Phase 32: Advanced Exploitation & Enterprise Network Lab Setup

Date: Mar 30, 2026

🔹 Enterprise Network Lab Configuration (VirtualBox)

Designed and deployed a segmented virtual network architecture to simulate a realistic, multi-tiered enterprise environment for advanced exploitation testing.

- **Infrastructure:** Configured multiple Virtual Machines on VirtualBox, utilizing the `172.16.x.x` private IP address space to establish proper routing and isolation.
- **Network Segmentation:**
    - **InternalZone:** A secure, internal corporate network housing endpoint operating systems (Windows 10, Windows 11) and specialized security distributions (**CommandoVM**).
    - **DMZ (Demilitarized Zone):** A semi-public subnetwork hosting deliberately vulnerable infrastructure (Metasploitable) to simulate exposed organizational web and application services.
    - **ExternalZone:** Simulated the untrusted public internet, serving as the staging

ground for the attacker machine (Kali Linux).

🔹 Practical Exploitation Exercises

Leveraged the segmented lab environment to execute a series of targeted attacks, progressing from initial access to custom payload delivery.

- **Practical 1: Linux OS Exploitation:** * Utilized the **Metasploit Framework** to scan, identify, and exploit vulnerabilities within the Linux targets hosted in the DMZ, successfully establishing an initial foothold.
- **Practical 2: Exploiting Samba (SMB) Vulnerability:** * Targeted a misconfigured Samba file-sharing service on a Linux endpoint. Executed an exploit to bypass authentication mechanisms, achieving unauthorized command execution and privilege escalation on the affected system.
- **Practical 3: Custom Payload Generation (MSFvenom):** * Generated a customized, standalone Windows payload using `msfvenom` specifically configured with a `windows/meterpreter/reverse_tcp` architecture.
    - Deployed the payload to the Windows targets within the InternalZone. Upon execution, the payload successfully established a reverse connection back to the Kali Linux listener in the ExternalZone, demonstrating a successful bypass of basic perimeter defenses and granting full remote command-line control (Meterpreter shell).

📅 Phase 33: Custom Tooling & Automation Scripting

Date: Mar 31, 2026

🔹 Developing `pentest_scan.sh`

Applied theoretical Bash scripting knowledge to create a practical, time-saving automation tool tailored for the initial reconnaissance phase of a penetration test.

- **Script Objective:** Automate the sequential execution of multiple enumeration tools against a target IP or domain, reducing manual overhead and standardizing output collection.
- **Features Implemented:**
    - **Tool Chaining:** Integrated essential discovery tools (e.g., `nmap` for port scanning, `gobuster` for directory brute-forcing, and `nikto` for web vulnerability scanning) into a unified execution flow.
    - **Output Consolidation:** Leveraged Bash I/O redirection operators (`>` and `>>`) to capture and append both standard output (`stdout`) and standard error (`stderr`) from all executing tools into a single, cohesive `.txt` report file.
    - **Dynamic Execution:** Configured the script to accept user-defined positional

parameters (e.g., passing the target IP as `$1`), allowing for flexible, on-the-fly execution against different targets.

- **Outcome:** Significantly accelerated the footprinting phase, ensuring that preliminary scan data is consistently formatted and automatically logged for later analysis.

🛠️ Technology Stack

- **OS:** Kali Linux, Ubuntu 24.04, Windows 10/11 Enterprise, **CommandoVM**, WSL 2
- **Development & Scripting:** React 18, Vite, JavaScript (JSX), Python 3.12, FastAPI, WebSockets, Motor (MongoDB Async), **Bash (Automation Scripting)**, UI/UX Animations
- **Cloud & Infrastructure:** Amazon Web Services (AWS EC2), Microsoft Azure (VM), Oracle Cloud Infrastructure (OCI), Vercel, MongoDB Atlas, Resend API
- **Android VAPT:** apktool, apksigner, ADB (Android Debug Bridge), Genymotion, keytool
- **Networking & Architecture:** OSI Model, TCP/IP, IPv4/IPv6, PCAP Analysis, ARP Spoofing, MAC Flooding, Active Directory (AD CS), Network Segmentation (DMZ, Internal, External)
- **Security Tools:** Nmap, Nessus, Pentest-Tools, Nikto, ssh-audit, swaks, OpenSSL, Metasploit, MSFvenom, Zphisher, MobSF, Burp Suite (Proxy, Intruder), Gobuster, Steghide, Param Miner, Ysoserial, Snort (IDS/IPS)
- **Concepts:** Cryptography, Web Security (CSP, Web Cache Poisoning, Insecure Deserialization, Cross-Site Scripting/XSS, SQL Injection/SQLi), API Integration, SSL Pinning Bypass, Domain Privilege Escalation, IDS/IPS Configuration
- **Virtualization:** VMware/VirtualBox (for Labs)

📝 Future Scope

- Finalize OCI server provisioning and backend migration for VulnForge.
- Expand `pentest_scan.sh` with conditional logic (e.g., running specific web tools only if port 80/443 is open).
- Deep dive into advanced Web Application Security (OWASP Top 10).

Internship Log maintained by Harmanjot Singh